

Final Project Report

Interactive Dashboard for WU's Collaborative Research Network

March 2025 – June 2025

Supervisor: Prof. Dr. Axel Polleres

Data Coach: Dr. Daniel Szöllösi

Team Members:

Wiktor Michal Uszko	h12230994	h12230994@s.wu.ac.at
Bolormaa Munkh-Erdene	h12233157	h12233157@s.wu.ac.at
Julius Leopold Moosmann	h12209031	h12209031@s.wu.ac.at

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Objectives of OPAL	4
1.3	Why OPAL?	5
1.4	Network visualization	5
2	Data Sources and Integration	5
2.1	Internal Data from PURE	5
2.2	External Sources	6
2.2.1	Author Affiliation Data	6
2.2.2	University Ranking Data	7
2.3	Data Enrichment and Matching	7
3	System Design and Implementation	8
3.1	The creation of nodes and edges	8
3.1.1	Initial inspection & design choices	8
3.1.2	Organisation lookup	8
3.1.3	Parsing contributor blocks	9
3.1.4	Exploring author $\times$ institute combinations	9
3.1.5	Node construction	9
3.1.6	Edge construction	9
3.1.7	Outputs & reproducibility	10
3.2	The script to download OpenAlex data	10
3.2.1	Reasoning for the data extraction	13
3.3	Main app	14
3.3.1	Libraries and functions	14
3.3.2	Settings for leaflet	15
3.3.3	Loading and preparing the data	15
3.3.4	User Interface	19
3.3.5	Server	20
3.3.6	Building the collaboration table for the chosen institute	28
3.3.7	Geocoding with a persistent cache	29
3.3.8	Slice the coordinate table into logical buckets	29
3.3.9	Composing the Leaflet map (m)	29
3.3.10	Return the reactive map	30

4	Challenges and Limitations	30
5	Future Work	31
5.1	Internally Proposed Improvements	31
5.2	Supervisor Feedback and Proposed Features	31
6	Conclusion	32
7	References	33
8	Sources and Tools	34
9	Appendix	35

Abstract

In the increasingly global landscape of research, it is important to understand and be able to visualize collaborative work of researchers in order to support interdisciplinary work and to improve visibility of institutions. At WU Vienna, the PURE system provides internal research metadata, but it lacks the functionalities for interactive exploration and benchmarking data globally. In our project, we try to address and improve on some of these limitations by developing an interactive R Shiny application that is designed to map and visualize research collaboration networks. This application, which we termed as OPAL (Overhauled, PURE, Access, Library), is made by using internal PURE data, and is enriched by external sources such as OpenAlex and the QS World University Rankings.

OPAL allows users to explore co-authorship patterns and institutional affiliations through dynamic network visualizations at the author and institutional levels. It offers filtering by publication count, geographical location, and university rankings, while overcoming some key challenges in data such as inconsistent institutional names and limited external metadata. In this project, we processed structured JSON data, enriched the initial data by exploring and selecting the appropriate external data sources, applied fuzzy matching techniques, and designed an approachable user interface using visNetwork library in R.

This final report for the project will include, in detail, the development process of the application, strategies for data integration we employed, the technical architecture, and the challenges encountered during the process of developing OPAL. We also demonstrate how OPAL could contribute to greater transparency in collaboration networks and support WU's journey to improve its global academic engagement.

1 Introduction

1.1 Motivation

In the highly interconnected and global research landscape, academic institutions can surely benefit from tools that will allow them to visualize and analyze collaboration patterns among researchers. International co-authorship is one of the key drivers for excellence in research, and the institutions that engage actively in global networks are more likely to produce more impactful work than the institutions that don't. Insights in collaborative research landscape could then support internal coordination, interdisciplinary collaboration, and improve visibility externally. These are all important for sustaining scientific competitiveness and strategic research planning.

The PURE system at WU Vienna functions as the central research information system in the institution. The system contains structured and reliable metadata on researchers, publication, and affiliations, and it's very useful for internal documentation and reporting. However, as we learned during our project while looking closely at it, PURE's existing visualization tools offer limited flexibility, with a room for improvement. While the collaboration network there focuses on internal relations and direct collaborations, it lacks in providing more interactive way to explore networks or benchmarking them internationally.

To this end, we developed an interactive R Shiny application, OPAL, to enhance WU's internal data with external sources and provide a more dynamic exploration of institutional and researcher networks.

1.2 Objectives of OPAL

The key objective of our project was to complement WU's internal research infrastructure through building a lightweight yet interactive application which would improve the way research collaborations are to be explored and analyzed. Mainly, our project aimed to:

- Enrich internal PURE data with publicly accessible external data to enhance the scope of collaboration network analysis.
- Develop a dynamic, user-friendly visualization tool at both the author and institutional levels.
- Offer filtering and customization options that go beyond the outputs of the current PURE system (such as filtering by institutional affiliation, publication count, or global university rankings).
- Design a scalable and transparent solution using open-source tools, such as R, Shiny, visNetwork, that can be expanded or adapted by future teams.

By meeting these objectives and goals, OPAL contributes to modernizing how research data is explored and used at WU Vienna, by moving towards a more interactive, globally minded, and insightful approach.

1.3 Why OPAL?

We named our application **OPAL** as an acronym for *Overhauled PURE Access Library*. This name reflects the project’s core aim: to upgrade and extend the capabilities of WU’s PURE system by enabling more dynamic access and interactive visualizations. Much like the gemstone opal, the app brings together diverse data sources to create a richer view of research collaborations. The name also captures the idea of clarity and insight, which aligns with our goal of making institutional networks more transparent and accessible.

1.4 Network visualization

The chosen way of visualizing the collaboration network in OPAL was heavily inspired by how the current PURE research network is visible in a web browser. The main issues we noticed in the current visualization is the lack of good filtering options and how quickly the graph can become cluttered with the more connected researchers. For this reason, there were significant changes we have decided to implement in order to make it visible. The network design was in some part informed by Chuang and Chen (2022) and Garner et al. (2018).

It was decided that the strength of a collaboration, which is the amount of co-written papers, will be marked by the thickness of the lines between the nodes of the authors. Due to drawing constraints all nodes will be kept at the same size.

2 Data Sources and Integration

The development of our app was centered around combining internal research metadata from WU’s PURE systems with relevant external datasets. This process has required us several phases of exploration, selection, and transformation as we have learned and evaluated the most suitable and feasible sources for enrichment of the initial PURE Data.

2.1 Internal Data from PURE

The starting point in our project was the structured JSON exports from the PURE system at WU. These included 4 different JSON files:

- **pure_research_output.json**: is the main source of publication metadata, including contributors, publication types, and affiliated institutions
- **pure_organisations.json**: structured metadata on WU’s organizational units and their UUIDs
- **pure_journals.json**: contains information about journals, including their titles, ISSNs, publishers, and classification labels.
- **pure_classification_schemes.json**: taxonomies and classifications used in PURE, useful for parsing subject areas, organizational roles, and name variants

We primarily used the `pure_research_output.json` and `pure_organisations.json` within our purposes in this project. The additional data from the other two files was explored but not integrated into our final application due to limited relevance for network construction.

We filtered and reshaped the data progressively. By extracting authors that are affiliated with WU, we matched them to their respective organizational units and built two main datasets that were the integral backbone for our project.

- **author_nodes.csv** and **author_edges.csv** for collaboration analysis

This transformation of the data enabled us the creation of our networks in the OPAL app. To optimize, we removed incomplete entries and reduced unnecessary noise, and kept only the relevant fields for visual exploration.

2.2 External Sources

To enrich the initial data provided to us from PURE, we explored several external data sources to retrieve institutional affiliations, university rankings, and other metadata about external co-authors. After exploring different options, we selected two primary external data sources. Firstly, we used OpenAlex to identify the institutions and the countries of the authors. Additionally, we used QS World University Rankings to add as a filtering option in our app. Below, we explain what other sources we have explored and why we have come to choose the specific ones to use.

2.2.1 Author Affiliation Data

In total, we considered three external research databases to get institutional data and country-level metadata for the authors:

- **ORCID** (orcid.org)

Although ORCID offered structured researcher profiles with persistent identifiers, we could not use it effectively in our project since PURE data doesn't have ORCID fields for both internal and external authors. Thus, matching through using ORCID was infeasible for our project.

- **Scopus** (scopus.com)

Scopus offers comprehensive metadata, but access to it is limited by strict API quotas and licensing terms. Thus, the rate limits made it unsuitable for the large-scale data enrichment in our project timeline.

- **OpenAlex** (openalex.org)

In the end, we selected OpenAlex as our main external research data source because of its accessibility, comparably generous API, and rich metadata on authors, institutions, and affiliations. It made it possible for

us to match researchers' publication titles to those in PURE and extract the associated institutional and country information.

2.2.2 University Ranking Data

One of our main goals of this project was to create a visualization that not only shows who has collaborated with whom, but also to see exactly how prominent the institutions that WU works with are. To this end, we have aimed to find a suitable university ranking data to integrate in our app. We looked at three major ranking websites and chose to integrate QS World University Rankings in our project. Our selection criteria included data availability, the relevance of the ranking to our project goals, the feasibility of integration.

- **Times Higher Education** ([timeshighereducation.com](https://www.timeshighereducation.com)):

Even though THE rankings are highly reputable, this source didn't offer openly accessible and structured dataset suitable for our use.

- **UniRank** (<https://www.unirank.org>):

While this ranking system offers very large global coverage filtered by continents, it focuses more on web presence and institutional popularity than academic quality, which made it a rather unsuitable choice for our project's goals.

- **QS World University Rankings** (<https://www.topuniversities.com>):

We selected to use QS as our university ranking dataset due to its high credibility and reputation, as well as its accessibly structured data format. It also provided very balanced methodology that accounts for multidimensional attributes, such as academic reputation, employer reputation, and research impact, which made it more aligned with our project goals.

2.3 Data Enrichment and Matching

We employed an algorithm to better match the authors from OpenAlex to the ones we had in the WU network. To do this, we downloaded all the profiles for a given name, then all the papers for all these profiles, and then checked which profile was linked to the research output with at least one paper. This greatly improved the accuracy of the findings and helped eliminate some uncertainty about the author's profile.

The OpenAlex data was used to identify the institutions of the researchers. This was then used to match these institutions to the ones present in the data set of the QS Ranking, and by extension, match the rankings to the universities from OpenAlex.

This was done with a fuzzy join in R. The accuracy was overall satisfactory but had certain limitations. For example, the University of Salzburg was always matched with the University of Klagenfurt instead. We

presume that it occurs because the linguistic similarity of these two is closer than the name for University of Salzburg, which is present in the QS data, that being Paris Lodron Universität Salzburg.

3 System Design and Implementation

The OPAL app includes two main parts used for its functionality, mainly dictated by the design of the Shiny library. The server is the part of the app that creates the logic necessary to run it. It includes all the functionality of the drawing of the graphs and the filters employed within the program. The other part of the code is focused on drawing graphical UI elements that allow the user to input the information and view the outputs of the app.

There are also different other sections, mostly tasked with the cleaning and pre-processing of the data and the leaflet initialization necessary for the institutional view. There also exist scripts that were used to download the data from the OpenAlex API. They are not a part of the app, and this data is not redownloaded again every time the app is launched. However, because they were very important in the development, it is important to remark on them

3.1 The creation of nodes and edges

3.1.1 Initial inspection & design choices

Exploring the JSON – We began by loading a handful of records in an R notebook, printing them with `str()` and `jsonlite::pretty()`. This revealed two critical facts:

- contributor blocks are heterogeneous – WU staff are wrapped in a person object, externals in an externalPerson object;
- authors can carry multiple organizations, stored as UUIDs in a list.

Identity strategy – To guarantee unique keys across both groups we decided on two prefixes right from the start:

- A_ + the internal person UUID for WU staff;
- EXT_ + the externalPerson UUID for any external contributor.

These IDs flow unchanged through every downstream step (network edges, geocoding, Shiny app).

3.1.2 Organisation lookup

Pure’s organisation tree is five to six levels deep but for collaboration we only cared about the institute and the university.

A helper function therefore walks up the .parents chain until it reaches the root node; the pair

`inst_uuid` $\rightarrow$ (`inst_name`, `root_name`)

is cached in a named list.

With that list, the parser can translate any UUID to a human-readable label instantly, without repeated joins.

3.1.3 Parsing contributor blocks

A streaming loop (`purrr :: map_dfr`) reads the `pure_research_output.json` file and emits one tidy row per contributor containing:

- publication UUID
- first / last name
- technical author ID (`A_...` or `EXT_...`)
- a list column with all organisation UUIDs linked to that contributor in that paper

External contributors frequently have no organisation attached; those rows are kept but the list element is set to NA.

Intermediate sanity checks were added after each 10 000 publications to spot malformed names or empty contributor lists early.

3.1.4 Exploring author $\times$ institute combinations

`tidyr :: unnest()` converts the list column into a long form so that every (author, institute) pair occupies its own row. A single left-join against the lookup table from the organisation injects `inst_name` and `uni_name`. At this point the dataframe already contains 90 % of what later becomes `author_nodes.csv`.

3.1.5 Node construction

Grouping by the technical author ID allows us to:

- count distinct publications (`pubs`),
- collect the unique institute names into a semicolon string (`institutes`),
- attach the previously determined `uni_name`.

The display label is simply “First Last”; missing first or last names are replaced by NA to avoid malformed labels.

3.1.6 Edge construction

For every publication with at least two contributors we generate the complete set of author pairs via `combn()`.

Pairs are canonicalised (alphabetically sorted) and then collapsed across the entire corpus, summing the number of joint papers into the edge weight. Because internal and external IDs are already disjoint (A_ vs EXT_) no further type column is required.

3.1.7 Outputs & reproducibility

The script produces two clean, ready-to-use tables:

- `author__nodes__new.csv` – one row per distinct contributor with display label, publication count, and institute / university information.
- `author__edges__new.csv` – one row per co-author pair with the number of shared papers as edge weight.

Because every person is addressed through an explicit prefix (A_ for internal staff, EXT_ for externals) and all organisational labels are resolved in a single, consistent lookup, both files can be regenerated at any time from the two original PURE exports without manual intervention.

3.2 The script to download OpenAlex data

The first script is used to obtain the author names for the purpose of searching them in the OpenAlex database. This is done as there is no specific identifier in the PURE database linking the researchers to OpenAlex. This will present a problem with duplicate names that will get resolved by matching the researchers' output with the output in PURE.

```
external_names <- nodes[grepl("^EXT_", nodes$id),]  
  
author_name <- external_names$label  
  
author_name <- author_name[author_name != "NA NA"]  
  
saveRDS(author_name, file = "author_name.rds")
```

The main script was a loop that run the logic over 1000 researcher names at a time. This was done in order to limit the possibility of a problem occurring and losing the pulled API data.

The script requires the papers (which are all PURE papers available) and the previously created `author__name` file.

Then, the url for the API is specified and the vectors used to store downloaded data are initialized.

```
library(httr)  
library(jsonlite)
```

```

papers <- readRDS("papers.rds")
author_name <- readRDS("author_name.rds")

author_name <- author_name[1:1000]

search_name <- URLencode(author_name, reserved = TRUE)
search_url <- paste0("https://api.openalex.org/authors?search=", search_name)

countries <- numeric(length(author_name))
uni <- numeric(length(author_name))

for (i in 1:length(search_url)) {
  response <- GET(search_url[i])
  search_results <-
    fromJSON(content(response, as = "text", encoding = "UTF-8"))
  res <- search_results$results

  results_for_name <- list()

  if (is.data.frame(res) && nrow(res) != 0) {
    for (j in 1:nrow(res)) {
      results_for_name[[length(results_for_name)+1]] <-
        fromJSON(content(GET(res$works_api_url[j]),
                           as = "text", encoding = "UTF-8"))$results
    }
  } else {
    results_for_name[[length(results_for_name)+1]] <- NA
  }

  for (h in 1:length(results_for_name)) {
    if(length(res) != 0) {
      if(length(which(results_for_name[[h]]$title
                      %in% papers$title.value)) != 0) {
        if(length(res$last_known_institutions[[h]]$display_name) != 0) {
          uni[i] <- (res$last_known_institutions[[h]]$display_name[1])
          countries[i] <- (res$last_known_institutions[[h]]$country_code[1])
        }
      }
    }
  }
}

```

```

    }
    else{
      uni[i] <- 0
      countries[i] <- 0
    }
    break
  }
}

rm(results_for_name)

}

downloaded_institutions <-
  data.frame(name = author_name, countries = countries, institutions = uni)

saveRDS(downloaded_institutions, file = "downloaded_institutions_1_100_test.rds")

```

To break down the logic of the main loop, the specific sub-loops will be explained:

```

response <- GET(search_url[i])
search_results <- fromJSON(content(response, as = "text", encoding = "UTF-8"))
res <- search_results$results

```

This chunk is responsible for the download of all profiles on OpenAlex that match the provided name. The result for this is a dataframe with the nrow equal to the number of profiles retrieved.

```

results_for_name <- list()
if (is.data.frame(res) && nrow(res) != 0) {
  for (j in 1:nrow(res)) {
    results_for_name[[length(results_for_name)+1]] <-
      fromJSON(content(GET(res$works_api_url[j]),
                          as = "text", encoding = "UTF-8"))$results
  }
} else {
  results_for_name[[length(results_for_name)+1]] <- NA
}

```

The next chunk is in charge of retrieving all articles available on OpenAlex written by all profiles that were retrieved in the previous step. The output is in the form of a list.

```
for (h in 1:length(results_for_name)) {
  if(length(res) != 0) {
    if(length(which(results_for_name[[h]]$title
                      %in% papers$title.value)) != 0) {
      if(length(res$last_known_institutions[[h]]$display_name) != 0) {
        uni[i] <- (res$last_known_institutions[[h]]$display_name[1])
        countries[i] <- (res$last_known_institutions[[h]]$country_code[1])
      }
    } else{
      uni[i] <- 0
      countries[i] <- 0
    }
    break
  }
}
}
```

The final loop is responsible for actually finding the institution of the researcher. It first runs through all of the outputs for each profile. It searches for the first profile index that contains an output that is available in the PURE data set. This is a way to ensure that the researcher in question is the specific researcher contained in PURE and not just a person with the same name. Then, after this has been identified, it retrieves the information about the institution and the country of the chosen profile and that concludes the entire step of the loop. The same process was then repeated for all available names.

After data, the vectors are merged into a dataframe and then saved in a .RDS file for further use.

3.2.1 Reasoning for the data extraction

The reasons for conducting the main data enrichment in such a way are somewhat specific to the sources used. The main reason was already described previously, OpenAlex is the best data source considering the scale of this project. With nearly 20 thousand external authors, one needs to access a lot of data. The API limits of Scopus or ORCID simply do not allow for this. This stands as the reason that OpenAlex was chosen.

The specific choice of of this method, one where both papers and the researcher profiles are used on OpenAlex is due to the existence of duplicate names. There are a lot of researchers that share a name and surname. This creates a situation in which one cannot clearly tell the current institution of the researcher just based on the name alone. That is why, while somewhat computationally demanding, it is decided to download all of

the papers created by each profile with a specific name and only match the ones that actually contributed to papers present in the PURE data set.

There was previously an attempt to create the links between researchers and institutions only based on names previously. This approach was a simpler version of the above and also utilized OpenAlex. The main issue that was faced is that when an institution was found, it was very seldom correct. The accuracy of this approach was strongly below 50%. The issue was tested by retrieving the institution of WU authors. It was noticed that a very low portion of them had assigned WU as the affiliated institution. This is mostly due to the fact that there were other researchers with the same name. OpenAlex sorts researcher profiles for an API pull using their own relevance score. While we are not certain how it is calculated, it most likely signifies the amount of output and collaboration of a researcher. This approach was always taking a person with the highest score, which was seldom the WU person.

A similar process was also conducted with Scopus, before encountering the issue with the API limit. The results of this work are described within the Appendix section. The main issue faced and discussed about Scopus was the occasional inconsistency of the website and the database. It is not certain why such a thing occurs, but sometimes there were instances of the cities of institutions not actually matching the real places where these institutions are located.

3.3 Main app

3.3.1 Libraries and functions

```
library(shiny)
library(visNetwork)
library(leaflet)
library(this.path)
library(dplyr)
library(scales)
library(tidygeocoder)
library(countrycode)
library(fuzzyjoin)

setwd(this.dir())
`%nin%` <- Negate(`%in%`)

clean_name <- function(x) {
  x <- tolower(x)
```

```

x <- gsub("[^a-z0-9\\s-]", "", x)
x <- gsub("\\s+", " ", x)
x <- trimws(x)
return(x)
}

```

The file begins by calling all the necessary libraries and defining two functions. *%nin%* is simply a negation of the *%in%* function present in base R. This is only used once in the file, but was defined for ease of reading. *clean_name()* is a function used for the cleaning of university names. It is used in the fuzzy joining of the OpenAlex universities to the ones from QS Ranking.

3.3.2 Settings for leaflet

```

# -----
# 0) Settings
# -----

cache_file <- "inst_coords_cache.rds"
vienna_name <- "Vienna University of Economics and Business"

```

3.3.3 Loading and preparing the data

This section is focused on loading the data and merging it in order to pass it into the server and app logic. This mostly focuses on the edges and nodes, created in a section above, and on the institution information downloaded from OpenAlex and QS.

```

# -----
# 1) Load & prepare data
# -----

nodes <- read.csv2("author_nodes_new.csv", stringsAsFactors = FALSE)
edges <- read.csv2("author_edges_new.csv", stringsAsFactors = FALSE)
institutions <- readRDS("downloaded_institutions_combined.rds")

qsranking <- read.csv("qsranking.csv")
qsranking <- subset(qsranking, select = c(index, rank_2025, rank_2024, institution_name, location))
qsranking$location <- countrycode(qsranking$location, origin = "country.name.en", destination = "iso2c")
colnames(qsranking) <- c('index', 'rank_2025', 'rank_2024', 'institutions', 'location')
colnames(edges) <- c("from", "to", "freq")

```

The data is loaded and preprocessed. For the *qsranking* file only a few columns are chosen. The country

names are encoded into iso2c (eg. Austria = AT) to make the use with the OpenAlex more comfortable. The columns are also renamed to make the structure more standardized.

```
nodes$author_id <- as.character(nodes$author_id)
edges$from <- as.character(edges$from)
edges$to <- as.character(edges$to)
```

The `as.character()` function is used in order to ensure these variables are strings, this is essential to the functioning of the app.

```
# attach external EXT_ IDs to institution records
external_ids <- nodes[grepl("^EXT_", nodes$id) & nodes$label!="NA NA", ]$id
institutions$id <- external_ids
```

IDs are reattached to the corresponding names in the *institutions* dataframe.

```
# build internal "WU" entries
internal_names <- nodes[grepl("^A_", nodes$id), ]$label
internal_ids <- nodes[grepl("^A_", nodes$id), ]$author_id
internal_institutions <- data.frame(
  label = internal_names,
  countries = "AT",
  institutions = "Vienna University of Economics and Business",
  id = internal_ids,
  stringsAsFactors = FALSE
)
```

A dataframe is created for the internal authors. This is necessary as an initial step to attach the data on institutions to the nodes dataframe.

```
# dedupe legacy institutions by (id,label)
institutions <- institutions %>%
  rename(label = name) %>%
  distinct(id, label, .keep_all = TRUE)

final_institutions <- bind_rows(internal_institutions, institutions) %>%
  distinct(id, label, .keep_all = TRUE)
```

The data is merged and filtered to only keep distinct IDs. This is not only done in order to avoid duplicates, but is necessary make the app run, as visNetwork does not accept any duplicate IDs.

```

# fix Unknown fields
final_institutions$countries [final_institutions$countries == 0] <- "Unknown"
final_institutions$institutions [final_institutions$institutions == 0] <- "Unknown"
colnames(final_institutions) <- c("label","countries","institutions","id")

# merge nodes + authors + institution info
nodes <- nodes %>%
  filter(label!="NA NA") %>%
  inner_join(final_institutions,
    by = c("id","label"),
    relationship = "many-to-many") %>%
  distinct(author_id, .keep_all = TRUE)

```

The zeros in the institutions dataframe are replaced with “Unknown”. After that, the dataframe is merged with the nodes using an inner join along the ID and name.

```

nodes$continent <- countrycode(sourcevar = nodes[, "countries"],
  origin = "iso2c",
  destination = "continent")

nodes$continent[which(is.na(nodes$continent))] <- "Unknown"

```

This chunk uses the *countrycode* library to add continents to the nodes based on the country that was added with the merge.

```

#qs implementation
nodes_qsraking <- subset(nodes, select = c(id, institutions, countries))

nodes_qsraking$institutions <- clean_name(nodes_qsraking$institutions)
qsraking$institutions <- clean_name(qsraking$institutions)

result <- stringdist_left_join(nodes_qsraking, qsraking, by = "institutions", method = "jw", max_dist = 1)
result <- result %>% group_by(id) %>% filter(countries == location | countries == "Unknown") %>% slice_max(rank_2025, n = 1)

#cleaning qsraking results
result$rank_2025 <- gsub('[+]', '', result$rank_2025)
result$rank_2024 <- gsub('[+]', '', result$rank_2024)

```

```

result$rank_2025_num <- as.numeric(sapply(strsplit(result$rank_2025, c('-')), "[", 1))
result$rank_2024_num <- as.numeric(sapply(result$rank_2024, function(x) {
  strsplit(x, '-')[[1]][1]
}))

```

This chunk shows how we use fuzzy join to match the names of the institutions from OpenAlex and QS. The basic idea is that the names of universities in both cases are cleaned and then a *jw* algorithm is run in order to check the distance between the strings. We only choose institutions if their countries match or are unknown and if the distance is less or equal to 0.2.

Lastly, we convert the rankings from strings to numeric.

```

#merging the cleaned results with nodes
result$rank <- (result$rank_2025_num + result$rank_2024_num)/2
result <- result[,c(1,12)]
nodes <- left_join(nodes, result, by = 'id')

```

The rank is calculated as the average of the ranks from 2025 and 2024. Then, the ranks are added to the nodes using a left join.

```

# collapse duplicate edges
edges <- edges %>%
  rowwise() %>%
  mutate(pair = paste(sort(c(from,to)), collapse=":")) %>%
  ungroup() %>%
  group_by(pair) %>%
  summarise(
    from = strsplit(pair,":")[[1]][1],
    to   = strsplit(pair,":")[[1]][2],
    freq = sum(freq),
    .groups = "drop"
  )

```

The edges are collapsed if they show connections between the same IDs and the frequencies are summed when this happens. This is not needed, but makes the graphs a lot more readable.

```

# build author tooltip
nodes$title <- paste0(
  "Name: ", nodes$label,
  "<br>Publications: ", nodes$pubs,

```

```

"<br>Country: ", nodes$countries,
"<br>Continent: ", nodes$continent,
"<br>Institution: ", nodes$institutions,
"<br>Rank: ", nodes$rank
)

```

The tooltip for the author nodes is created from the variables of the *nodes* dataframe.

3.3.4 User Interface

```

# -----
# 3) UI
# -----

ui <- fluidPage(
  titlePanel(div(img(src="logo.png"), "Welcome to OPAL")),
  sidebarLayout(
    sidebarPanel(
      radioButtons("view_mode", "Top-level view:",
        choices = c("Author view", "Institution view"),
        inline = TRUE),
      uiOutput("edge_buttons"),
      uiOutput("dynamic_node_picker"),
      uiOutput("dynamic_net_switch"),
      uiOutput("inst_selector"),
      uiOutput("dynamic_slider"),
      uiOutput("rank_slider"),
      uiOutput("cont_checkbox")
    ),
    mainPanel(
      conditionalPanel("input.view_mode=='Author view'",
        visNetworkOutput("filteredGraph", height="550px")),
      conditionalPanel("input.view_mode=='Institution view'",
        leafletOutput("inst_map", height="550px"))
    )
  )
)

```

The UI will not be discussed here as most of its features are store in dynamic UI elements defined in the

server. The code just serves as a view as to how the app is structured.

3.3.5 Server

```
# -----  
# 4) Server  
# -----  
server <- function(input, output, session) {}
```

All following code chunks should be in this server function, but for the sake of readability they were separated.

```
# -- edge drawing logic selection (connected vs. further) --  
output$edge_buttons <- renderUI({  
  req(input$view_mode=="Author view", input$selected_node)  
  radioButtons("edge_mode", "Edge-drawing logic:",  
    choices=c("Simple connections", "Further connections"),  
    selected="Simple connections")  
})
```

The first element is the output element that creates the input for the simple/further connections.

```
# -- dynamic author vs. institution selector --  
output$dynamic_node_picker <- renderUI({  
  if (input$view_mode=="Author view") {  
    selectInput("selected_node", "Select an author:",  
      choices = nodes$label, selected = nodes$label[1])  
  } else {  
    # only show institutions with coords  
    avail <- intersect(  
      unique(nodes$institutions),  
      inst_coord %>% filter(!is.na(latitude), !is.na(longitude)) %>% pull(address)  
    )  
    selectizeInput("selected_inst", "Select an institution:",  
      choices = sort(avail),  
      selected = sort(avail)[1],  
      options = list(server=TRUE))  
  }  
})
```

This element is controlled by the input of the top view. If we choose the top view of authors, this will

create the *selected_node* input, that is used to display the networks of different researchers. If the top view is institutions, this will create the *selected_inst* input, which is used to select institutions to show their connections on the map.

```
output$cont_checkbox <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  checkboxGroupInput(
    "cont_filter", "Show continents:",
    choices = sort(unique(nodes$continent)),
    selected = sort(unique(nodes$continent))
  )
})
```

This creates the filter that allows to show nodes by continents. It requires the author view to be selected in order to show and function.

```
# -- institution rank filter (author view only) --
output$rank_slider <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  sliderInput("rank_filter", "Max Rank:",
    min = min(na.omit(nodes$rank)), max = max(na.omit(nodes$rank)),
    value = max(na.omit(nodes$rank)))
})
```

This creates the filter that allows to show nodes by institution rank. It will completely ignore and not filter nodes without institution rank. It requires the author view to be selected in order to show and function.

```
# -- show-institutes filter (author view only) --
output$inst_selector <- renderUI({
  req(input$view_mode=="Author view", input$selected_node)
  ego_id <- nodes$author_id[nodes$label==input$selected_node]
  ce <- subset(edges, from==ego_id|to==ego_id)
  ids <- unique(c(ce$from,ce$to,ego_id))
  insts <- nodes %>% filter(id%in%ids) %>% pull(institutions) %>% unique()
  checkboxGroupInput("inst_filter","Show institutions:",
    choices=insts, selected=insts)
})
```

This creates the filter that allows for the display of nodes based on the institution. It detects and draws the list of institutions dynamically to only those that are available on the drawn network. It requires the author

view to be selected in order to show and function.

```
# -- author-centric network switch --
output$dynamic_net_switch <- renderUI({
  req(input$view_mode=="Author view")
  radioButtons("net_mode", "Author-centric network:",
    choices=c("People network", "Institution network"),
    selected="People network")
})
```

This output creates an input element if the top view of authors is selected. It allows for the selection of a network showing the network of either authors or institutions connected to the selected nodes. The logic in how this is achieved is explained below.

```
# -- slider: pubs vs. collabs --
output$dynamic_slider <- renderUI({
  if (input$view_mode=="Author view") {
    sliderInput("pubs_filter", "Max Publications:",
      min=min(nodes$pubs),
      max=max(nodes$pubs),
      value=max(nodes$pubs),
      step=1)
  } else {
    req(input$selected_inst)
    maxc <- inst_edges %>%
      filter(inst_from==input$selected_inst|inst_to==input$selected_inst) %>%
      pull(freq) %>% max(na.rm=TRUE)
    sliderInput("collab", "Max Collaborations:",
      min=1, max=maxc, value=maxc, step=1)
  }
})
```

The above chunk shows the pubs/collabs slider. Based on the selected top view, it will change the slider to either show author publications or institution collaborations.

```
# --- render author view network ---
output$filteredGraph <- renderVisNetwork({
  req(input$view_mode=="Author view", input$selected_node)
```

```

# extract ego-network for the selected author
ego_id <- nodes$author_id[nodes$label==input$selected_node]
ce <- subset(edges, from==ego_id|to==ego_id)
ids <- unique(c(ce$from,ce$to,ego_id))
cn <- subset(nodes, id %in% ids)

pn <- subset(cn, id == ego_id)
fn <- subset(cn, id != ego_id)

fur_e <- subset(edges,
                 (from %in% fn$id | to %in% fn$id) &
                 !(from == ego_id | to == ego_id)
)

if(input$edge_mode == "Simple connections") {
  ae <- ce
} else if (input$edge_mode == "Further connections") {
  ae <- (rbind(ce, fur_e))
}

fn <- fn %>% filter(institutions %in% input$inst_filter)
fe <- subset(ae, from%in%fn$id & to%in%fn$id)

if (input$net_mode=="Institution network") {
  # keep one central node + one per partner institution
  main_node <- pn

  fn <- fn %>% filter(pubs <= input$pubs_filter)
  fn <- fn %>% filter(continent %in% input$cont_filter)
  fn <- fn %>% filter(rank <= input$rank_filter | is.na(rank))

  inst_nodes <- fn[fn$id!=ego_id,] %>%
    distinct(institutions,.keep_all=TRUE) %>%
    mutate(
      id = paste0("inst_",institutions),

```

```

    label = institutions,
    title = paste0("Institution: ",institutions)
  )
  fn <- bind_rows(main_node,inst_nodes)
  fe <- inst_nodes %>% transmute(from=ego_id,to=id,freq=1)
} else if (input$net_mode=="People network") {
  main_node <- pn

  fn <- fn %>% filter(pubs <= input$pubs_filter)
  fn <- fn %>% filter(continent %in% input$cont_filter)
  fn <- fn %>% filter(rank <= input$rank_filter | is.na(rank))

  fn <- bind_rows(main_node,fn)
  fe <- ae
}

if (nrow(fe)>0) {
  fe <- fe %>%
    mutate(
      width = scales::rescale(freq,to=c(1,10)),
      title = paste("Collaborations:",freq)
    )
}

visNetwork(fn,fe) %>%
  visLayout(randomSeed=1) %>%
  visEdges(smooth=TRUE) %>%
  visPhysics(
    solver="forceAtlas2Based",
    forceAtlas2Based=list(
      gravitationalConstant=-60,
      centralGravity=0.015,
      springLength=230,
      springConstant=0.08
    ),
    stabilization=FALSE

```

```

) %>%
visEvents(stabilizationIterationsDone=
  "function(){ this.fit({animation:false}); }")
})

```

And this chunk is the most important of the entire app. It is fully responsible for the creation of the graph given just the edges and nodes and all other user inputs like filters.

It first creates the `ego_id`, which is the ID of the selected node. Then using that, it finds all edges connected to the `ego_id` and by extension all nodes that are connected to the selected node. These nodes are the *connected nodes* or *cn* for short. Then it initializes the *primary node* (*pn*), and the *filtered nodes* (*fn*). Lastly, it creates the *further edges* (*fur_e*), based on the edges connecting the edges that connect all drawn nodes except for the *primary node*.

The first if-statement sees if the input is either only simple or further connections and uses either only *connected edges* or the bind of *connected edges* and *further edges*. Then it filters the *filtered nodes* by the institution filter and initializes *filtered edges*.

The second if-statement is tasked with dealing with the selection of either the people network or the institution network. If the institution view was chosen by the user, it will begin by filtering *filtered nodes* based on user input. Then it creates the *inst_nodes* by collapsing all institutions assigned to authors, keeping only distinct ones. After that, it binds them to the primary nodes. If the author view was chosen, it will filter the nodes the same way it did in the previous step. After the filter, it will bind the *primary node* to the *filtered nodes*.

The last if-statement checks if any edges are drawn and if yes, then it rescales them to have a more sensible width.

The last part is drawing the graph. It uses some parameters to make the graph look more stable and limit the physics to allow for better performance.

```

# --- render institution view map ---
output$inst_map <- renderLeaflet({
  req(input$view_mode=="Institution view", input$selected_inst)
  centre <- input$selected_inst

  link_tbl <- inst_edges %>%
    filter((inst_from==centre|inst_to==centre) & freq<=input$collab) %>%
    mutate(partner=ifelse(inst_from==centre,inst_to,inst_from))

  # on-the-fly geocode any new partners

```

```

all_needed <- unique(c(centre,link_tbl$partner))
to_geo     <- setdiff(all_needed,
                      inst_coord %>% filter(!is.na(latitude)) %>% pull(address))
if (length(to_geo)>0) {
  newc <- geocode(
    data.frame(address=to_geo,stringsAsFactors=FALSE),
    address,method="osm",lat="latitude",long="longitude"
  )
  inst_coord <-< bind_rows(inst_coord,newc) %>%
    distinct(address,.keep_all=TRUE)
  saveRDS(inst_coord,cache_file)
}

pts      <- inst_coord %>%
  filter(address%in%all_needed & !is.na(latitude))
centre_pt <- pts %>% filter(address==centre)
partner_pts <- pts %>% filter(address!=centre)
vienna_pt <- partner_pts %>% filter(address==vienna_name)
other_pts <- partner_pts %>% filter(address!=vienna_name)

m <- leaflet() %>% addTiles()

# center marker
if (nrow(centre_pt)>0) {
  m <- m %>% addCircleMarkers(
    lng=centre_pt$longitude,lat=centre_pt$latitude,
    popup=centre,radius=10,color="red",fillOpacity=0.9
  )
}

# other partners
if (nrow(other_pts)>0) {
  m <- m %>% addCircleMarkers(
    lng=other_pts$longitude,lat=other_pts$latitude,
    popup=paste0(
      other_pts$address,"<br>Collaborations: ",

```

```

    link_tbl$freq[match(other_pts$address,link_tbl$partner)]
  ),
  radius=6,color="steelblue",fillOpacity=0.7
)
}

# Vienna highlight
if (nrow(vienna_pt)>0) {
  m <- m %>% addCircleMarkers(
    lng=vienna_pt$longitude,lat=vienna_pt$latitude,
    popup=paste0(
      vienna_name,"<br>Collaborations: ",
      link_tbl$freq[match(vienna_name,link_tbl$partner)]
    ),
    radius=8,color="orange",fillOpacity=0.9
  )
}

# draw lines
if (nrow(centre_pt)>0 && nrow(partner_pts)>0) {
  for (i in seq_len(nrow(partner_pts))) {
    p <- partner_pts$address[i]
    col <- if (p==vienna_name) "orange" else "gray"
    w <- if (p==vienna_name) 3 else 1
    op <- if (p==vienna_name) 0.8 else 0.6
    m <- m %>% addPolyLines(
      lng=c(centre_pt$longitude,partner_pts$longitude[i]),
      lat=c(centre_pt$latitude,partner_pts$latitude[i]),
      color=col,weight=w,opacity=op
    )
  }
}

# auto-zoom
if (nrow(centre_pt)>0 && nrow(partner_pts)>0) {
  all_lng <- c(centre_pt$longitude,partner_pts$longitude)

```

```

all_lat <- c(centre_pt$latitude, partner_pts$latitude)
m <- m %>% fitBounds(
  lng1=min(all_lng),lat1=min(all_lat),
  lng2=max(all_lng),lat2=max(all_lat)
)
} else if (nrow(centre_pt)>0) {
  m <- m %>% setView(
    lng=centre_pt$longitude,lat=centre_pt$latitude,zoom=4
  )
} else if (nrow(partner_pts)>0) {
  m <- m %>% fitBounds(
    lng1=min(partner_pts$longitude),
    lat1=min(partner_pts$latitude),
    lng2=max(partner_pts$longitude),
    lat2=max(partner_pts$latitude)
  )
}

m
})

```

3.3.6 Building the collaboration table for the chosen institute

centre is used to signify the institute picked in the drop-down. Then **inst_edges** (pre-computed institute-to-institute links) is trimmed twice:

- Keep only rows where either end equals **centre**.
- Discard rows whose collaboration count (**freq**) is above the slider value **input\$collab**.

Lastly, **mutate()** adds a helper column **partner** that always contains “the other side” of the link.

inst_from	Inst_to	freq	partner
centre	A	5	A
B	centre	3	B

The resulting **tink_tbl** stores the data to be plotted and how thick each connecting line should be.

3.3.7 Geocoding with a persistent cache

The objective is to turn institute names into lat/long pairs, but never hit the OSM API more than once per name.

Algorithm:

1. Compose `all_needed = centre`.
2. Compare to `inst_coord`, a disk-backed cache that already stores every location queried so far.
3. Only the true set difference (`to_geo`) is sent to `tidygeocoder::geocode(method = "osm")`.
4. New rows are merged back into `inst_coord` and saved to `inst_coords_cache.rds`.

3.3.8 Slice the coordinate table into logical buckets

Once all coordinates are ready, four subsets are carved out: - `centre_pt` – one row, the focal institute

- `partner_pts` – every partner with coordinates
- `vienna_pt` – `partner_pts` where `address == vienna_name`
- `other_pts` – `partner_pts` minus Vienna

This segregation is the basis of the colour, radius, and tooltip logic later on.

3.3.9 Composing the Leaflet map (m)

Base layer

`leaflet()` lays down the standard OpenStreetMap raster tiles.

Point markers

Group	Colour & size	Tooltip content
<code>centre_pt</code>	red, radius 10	institute name
<code>other_pts</code>	steel-blue, radius 6	Name + line break + “Collaborations: n”
<code>vienna_pt</code>	orange, radius 8	same template

Connection lines

A `for`-loop walks over every partner row:

- Grey, weight 1 for ordinary links.
- Orange, weight 3 for Vienna.

Opacity is also bumped for the highlighted line.

Each polyline is drawn from the centre’s coordinates to the partner’s.

3.3.10 Return the reactive map

The fully configured Leaflet object `m` is returned from `renderLeaflet()`.

Because every step above depends on reactive inputs (`view mode`, `selected institute`, `slider threshold`, `Vienna name`, `cache state`), **Shiny will rerun the entire pipeline** whenever any of those values change, keeping the map in perfect sync with user interaction.

4 Challenges and Limitations

We have encountered a range of challenges while working on this project, and in this section, we will address the primary ones we have faced and mention the limitations of our app.

1. **The limited coverage in OpenAlex:** The OpenAlex database does not contain a lot of the researchers present in the PURE database. Around 60% of all PURE authors could not be found on OpenAlex or had no affiliation with any institution on OpenAlex. This influenced the completeness of both the author and institutional level networks.
2. **Fuzzy Matching and Institutional ambiguity:** Fuzzy join and the different algorithms of it do face an issue when matching some of the university names. Overall, this is not a huge problem but can be seen as a limitation and maybe a different approach to the pre-processing of the strings.
3. **API Constraints and Missed Opportunities with Scopus:** The OpenAlex API issues could have been amended if the Scopus API was used. Upon manual inspection, Scopus does seem to have better results in at least a few cases. However, the free Scopus API key is very restrictive and would make the creation of the dataset too difficult. Institutional access to Scopus could improve the results in future iterations.
4. **Performance issues in Shiny:** The R Shiny environment struggles with some high-intensity operations and makes the UI stutter and slow down. Particularly, the use of VisNetwork package applies a physics simulation by default, which could become performance intensive as the network size increases. For us, this led to occasional freezing of the app in a case of highly connected nodes creating UI lags. As a result, we have chosen not to include the highly connected individuals and institutions in our demo, as they could overload the app.
5. **No internal collaboration view or Sub-Organization Centering:** As of now, OPAL focuses mainly on showing external collaborations of WU researchers. There is no dedicated view to visualize and explore the internal collaboration patterns within WU, such as connections between sub-organizations and departments. As suggested by our supervisor, implementing a sub-org-centered or “outside-in” view could largely improve OPAL’s usefulness.

5 Future Work

While our work with building up OPAL provided a functional and insightful prototype for exploring research collaborations at WU, there are several opportunities to improve it further both in technical and user-experience aspects. In this section, we will divide these potential improvements in two parts. First, we discuss the enhancements based on our internal team goals we have agreed on during our work. Next, we also incorporate the features and improvements suggested by our supervisor after the final presentation.

5.1 Internally Proposed Improvements

- **Gain Access to the Scopus API on a better license:** Doing this would significantly increase data coverage and retrieval accuracy and will improve speed and scalability of the data enrichment.
- **Develop a way to match names which have no information online to some institution:** One option could include using adjacency to other co-authors with known affiliations to infer missing information.
- **Enrich PURE dataset with ORCID identifiers where possible:** Linking author entries to ORCID would help improve consistency and match quality.
- **Develop more useful filters to gain required insights more easily:** This would allow users to extract meaningful collaboration patterns more easily.
- **Look into displaying statistics on collaboration in a more accessible manner:** Adding simple numeric insights, such as number of publications, most collaborative authors, in a more accessible format to the user could enhance their experience using the app greatly.
- **Explore the possibility of hosting or different implementation:** This could improve performance, particularly for large datasets.
- **Improve the user interface and experience (UI/UX) where possible:** Increased UI/UX through smoother navigation, clearer layout, and more responsiveness would make the app much more approachable for users with varying backgrounds.

5.2 Supervisor Feedback and Proposed Features

- **Sub-organization-centered view:** Aggregation at WU sub-organization level (e.g., departments or institutes).
- **Internal collaboration view:** Within-WU collaboration view by sub-organization.
- **“Outside-in” visualization:** External institutions in the center, showing who/which sub-organization has collaborated from WU with which university or country.

- **Fallback mechanism for incomplete data:** Think of how to “fallback” to UniRank when QS is not available to avoid losing any connections.
- **Scalability improvements for highly connected entities:** Pre-materialize certain values/aggregations for people or institutions with many connections.
- **Topic-based filters and subject specific rankings:** In collaboration with PURE 2 group, explore into adding QS ranking per topics/programs and adding research topics.

6 Conclusion

Our project working on OPAL has aimed to improve WU’s ability to understand and visualize its research collaboration network better by integrating both internal and external data sources into an interactive R Shiny application. We combined structured data from the PURE system with external data from OpenAlex and QS World University Rankings, and developed a prototype that has potential to offer filterable, dynamic, and intuitive network visualizations in both author and institutional levels.

During the process of building OPAL, our team has faced several challenges, such as inconsistencies in data, external data coverage limitations, and performance issues in R Shiny. Still, we have tackled these challenges to the best of our abilities for the given project timeframe and have successfully built an adaptable and buildable application that potentially lays the groundwork for future improvements and iterations in a larger institutional use at WU Vienna.

We believe that OPAL shows promising potential for future developments, such as incorporating internal collaboration views, improvements in matching, larger filtering options, and better user interface and experience. With these potential enhancements, OPAL could become a strong tool for research planning, collaboration mapping, and decision making at the university.

This project has taught us many meaningful lessons and provided valuable insights into technical aspects of data visualization in academia and the practical considerations of working with available research databases. We hope and believe that OPAL could inspire WU and other institutions in seeking better understanding of their collaborative research landscape and help steer them into a direction of more visible, transparent and globally focused future of academic research collaboration.

7 References

- Chuang, Yung-Ting, and Yi-Hsi Chen. 2022. “Social Network Analysis and Data Visualization of MIS International Collaboration in Taiwan.” *Library Hi Tech* 40 (5): 1422–58.
- Dong-Hui, Yang, Yan Wang, Yu Tian, and Xueyu Liu. 2020. “Macro-Level Collaboration Network Analysis and Visualization with Essential Science Indicators: A Case of Social Science.” *Malaysian Journal of Library & Information Science* 25 (1): 121.
- Garner, Jon, Alan L Porter, Andreas Leidolf, and Michelle Baker. 2018. “Measuring and Visualizing Research Collaboration and Productivity.” *J. Data Inf. Sci.* 3 (1): 54–81.
- Jing, Ming, Xueqing Li, and Li Zhang. 2019. “Interactive Temporal Display Through Collaboration Networks Visualization.” *Information Visualization* 18 (2): 268–80.
- Merino, Leonel, Dominik Seliner, Mohammad Ghafari, and Oscar Nierstrasz. 2016. “Communityexplorer: A Framework for Visualizing Collaboration Networks.” In *Proceedings of the 11th Edition of the International Workshop on Smalltalk Technologies*, 1–9.
- Perianes-Rodriguez, Antonio, Ludo Waltman, and Nees Jan Van Eck. 2016. “Constructing Bibliometric Networks: A Comparison Between Full and Fractional Counting.” *Journal of Informetrics* 10 (4): 1178–95.
- Shneiderman, Ben. 2003. “The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations.” In *The Craft of Information Visualization*, 364–71. Elsevier.

8 Sources and Tools

Internal Data Source

- PURE System (WU Vienna) – WU’s internal research system provided the core metadata, including publication records, author affiliations, and organizational units. The data was accessed through exported JSON files:
 - `pure_research_output.json`
 - `pure_organisations.json`
 - `pure_journals.json`
 - `pure_classification_schemes.json`

External Data Sources

- OpenAlex: <https://openalex.org/>
- QS World University Rankings: <https://www.topuniversities.com>
- ORCID: <https://orcid.org/>
- Scopus: <https://www.scopus.com/>
- UniRank: <https://www.unirank.org/>
- Times Higher Education: <https://www.timeshighereducation.com/>

Technical Tools and Libraries

- R: <https://www.r-project.org/>
- Shiny: <https://shiny.posit.co/>
- leaflet: <https://leafletjs.com/>
- visNetwork: <https://datastorm-open.github.io/visNetwork/>
- tidygeocoder: <https://cran.r-project.org/web/packages/tidygeocoder/vignettes/tidygeocoder.html>
- Github was used for collaborative coding and tracking for the team
- ChatGPT and similar AI tools were used to speed up the code refinement and error fixing

9 Appendix

Github Repository

<https://github.com/Wixo7/OPAL>

PURE Data Processing

```
pkgs <- c("jsonlite", "purrr", "dplyr", "tidyr", "stringr", "tibble", "readr")
install <- setdiff(pkgs, rownames(installed.packages()))
if (length(install)) install.packages(install)
invisible(lapply(pkgs, library, character.only = TRUE))

base <- "C:/Users/Julius/Desktop/OPAL/Data integration/Data from jupyter"
org_file <- file.path(base, "pure_organisations.json")
ro_file <- file.path(base, "pure_research_output.json")

org_tbl <- fromJSON(org_file, simplifyVector = FALSE)$items |>
  map_dfr(\(o) {
    vals <- map_chr(o$name$text, "value")
    locs <- map_chr(o$name$text, "locale")
    name <- vals[match("en_GB", locs)] %||% vals[1]
    tibble(inst_id = o$uuid, inst_label = name)
  })

ro_items <- fromJSON(ro_file, simplifyVector = FALSE)$items

author_long <- map_dfr(ro_items, function(item) {

  if (is.null(item$contributors) || length(item$contributors) == 0)
    return(NULL)

  orgs_pub <- if (length(item$organizations))
    map_chr(item$organizations, "uuid", .default = NA_character_)
  else NA_character_

  map_dfr(item$contributors, function(c) {
    tibble(
```

```

    pub_id      = item$uuid,
    author_id    = c$person$uuid %||% NA_character_,
    first_name   = c$name$firstName %||% NA_character_,
    last_name    = c$name$lastName %||% NA_character_,
    inst_ids     = list(orgs_pub) # List-Spalte (evtl. mehrere)
  )
})
})

author_nodes <- author_long |>
  group_by(author_id, first_name, last_name) |>
  summarise(
    publications = n_distinct(pub_id),
    inst_ids     = unique(unlist(inst_ids)),
    .groups      = "drop"
  ) |>
  mutate(
    label      = paste(first_name, last_name),
    institutes = map_chr(inst_ids, \(v)
      str_c(org_tbl$inst_label[match(v, org_tbl$inst_id)],
            collapse = "; ")
  ) |>
  select(author_id, label, first_name, last_name,
         institutes, publications)

author_edges <- author_long |>
  distinct(pub_id, author_id) |>
  group_by(pub_id) |>
  filter(n() >= 2) |>
  summarise(pairs = list(combn(author_id, 2, simplify = FALSE)),
            .groups = "drop") |>
  unnest(pairs) |>
  mutate(source = map_chr(pairs, 1),
         target  = map_chr(pairs, 2)) |>
  count(source, target, name = "weight")

```

```

inst_edges <- author_long |>
  select(pub_id, inst_ids) |>
  unnest(inst_ids) |>
  distinct() |>
  group_by(pub_id) |>
  filter(n() >= 2) |>
  summarise(pairs = list(combn(inst_ids, 2, simplify = FALSE)),
            .groups = "drop") |>
  unnest(pairs) |>
  mutate(source = map_chr(pairs, 1),
         target = map_chr(pairs, 2)) |>
  count(source, target, name = "weight") |>
  filter(source != target)

inst_nodes <- org_tbl |>
  semi_join(inst_edges, by = c("inst_id" = "source")) # jetzt korrekt

readr::write_csv2(author_nodes, file.path(base, "author_nodes.csv"))
readr::write_csv2(author_edges, file.path(base, "author_edges.csv"))

readr::write_csv2(inst_nodes, file.path(base, "institute_nodes.csv"))
readr::write_csv2(inst_edges, file.path(base, "institute_edges.csv"))

message("Vier CSV-Dateien erstellt - bereit fürs Netzwerk-Tool!")

```

ORCID ID check

```

library(jsonlite)
library(purrr)
library(dplyr)

items <- fromJSON(
  "C:/Users/Julius/Desktop/OPAL/Data integration/Data from jupyter/pure_research_output.json",
  simplifyVector = FALSE
)$items

```

```

has_orcid <- map_lgl(items, function(pub) {
  any(map_lgl(pub$contributors, function(c) {

    int_orcid <- !is.null(c$person$identifiers) &&
      any(map_chr(c$person$identifiers, "typeDiscriminator") == "ORCID")

    ext_orcid_list <- !is.null(c$externalPerson$identifiers) &&
      any(map_chr(c$externalPerson$identifiers, "typeDiscriminator") == "ORCID")

    ext_orcid_field <- !is.null(c$externalPerson$orcid)

    contrib_field <- !is.null(c$orcid)

    int_orcid || ext_orcid_list || ext_orcid_field || contrib_field
  }))
})

mean(has_orcid)
sum(has_orcid)

```

Scopus Sample

```

if (!requireNamespace("httr", quietly = TRUE)) {
  install.packages("httr")
}

if (!requireNamespace("jsonlite", quietly = TRUE)) {
  install.packages("jsonlite")
}

library(httr)
library(jsonlite)

base_path <- "C:/Users/Julius/Desktop/OPAL/Data integration/Data from jupyter"
nodes_in <- file.path(base_path, "author_nodes_new.csv")

sample_size <- 100

```

```

output_file <- file.path(base_path, "external_authors_sample_scopus.csv")

scopus_api_key <- "fe6385c29022b7b8b8c8217cddfff7de"

fetch_scopus_for_one <- function(first_name, last_name, api_key) {
  raw_query <- paste0("AUTHLASTNAME(", last_name, ") AND AUTHFIRST(", first_name, ")")
  base_url <- "https://api.elsevier.com/content/search/author"

  response <- GET(
    url = base_url,
    query = list(query = raw_query, count = 1),
    add_headers(
      `X-ELS-APIKey` = api_key,
      Accept          = "application/json"
    )
  )

  if (status_code(response) != 200) {
    warning(sprintf("HTTP-Fehler %d beim Abruf für '%s %s'.",
                    status_code(response), first_name, last_name))
    return(data.frame(
      scopus_id      = NA_character_,
      affiliation_name = NA_character_,
      affiliation_country = NA_character_,
      stringsAsFactors = FALSE
    ))
  }

  text_content <- content(response, as = "text", encoding = "UTF-8")
  parsed_json <- fromJSON(text_content, flatten = TRUE)
  entries <- parsed_json[["search-results"]][["entry"]]

  if (is.null(entries) || nrow(entries) == 0) {
    message(sprintf("Kein Scopus-Treffer für '%s %s'.", first_name, last_name))
  }
}

```

```

return(data.frame(
  scopus_id      = NA_character_,
  affiliation_name = NA_character_,
  affiliation_country = NA_character_,
  stringsAsFactors = FALSE
))
}

e <- entries[1, ]

full_id <- e$`dc:identifier` # z.B. "AUTHOR_ID:57196429313"
scopus_id <- if (!is.null(full_id) && grepl(":", full_id)) {
  sub(".*:(.*)$", "\\1", full_id)
} else {
  NA_character_
}

aff_name <- if (!is.null(e$`affiliation-current.affiliation-name`)) e$`affiliation-current.affiliation-name`
aff_country <- if (!is.null(e$`affiliation-current.affiliation-country`)) e$`affiliation-current.affiliation-country`

return(data.frame(
  scopus_id      = scopus_id,
  affiliation_name = aff_name,
  affiliation_country = aff_country,
  stringsAsFactors = FALSE
))
}

if (!file.exists(nodes_in)) {
  stop(paste0("Datei '", nodes_in, "' nicht gefunden. Bitte Pfad prüfen."))
}

all_authors <- read.csv2(nodes_in, stringsAsFactors = FALSE)

cat("Spalten in all_authors:\n")

```

```

print(names(all_authors))

external_authors <- all_authors[grepl("^EXT_", all_authors$author_id), ]

cat("Nach Filtern EXT_ hat external_authors Spalten:\n")
print(names(external_authors))

external_authors <- external_authors[
  !is.na(external_authors$first_name) & external_authors$first_name != "" &
  !is.na(external_authors$last_name) & external_authors$last_name != "",
]

cat("Struktur von external_authors nach Entfernen von NA/\":\n")
print(str(external_authors))

n_ext <- nrow(external_authors)
if (n_ext == 0) {
  stop("Es wurden keine externen Autoren gefunden (author_id beginnt mit 'EXT_').")
}
if (n_ext < sample_size) {
  warning(sprintf(
    "Es gibt nur %d externe Autoren, sample_size = %d ist zu groß. sample_size wird auf %d reduziert.",
    n_ext, sample_size, n_ext
  ))
  sample_size <- n_ext
}

set.seed(1234)
sampled_ext <- external_authors[sample(n_ext, size = sample_size), ]

results_list <- vector("list", sample_size)

for (i in seq_len(sample_size)) {
  first <- sampled_ext$first_name[i]
  last <- sampled_ext$last_name[i]

```

```

info_df <- fetch_scopus_for_one(first, last, scopus_api_key)

if (is.null(info_df)) {
  info_df <- data.frame(
    scopus_id          = NA_character_,
    affiliation_name    = NA_character_,
    affiliation_country = NA_character_,
    stringsAsFactors   = FALSE
  )
}

combined <- data.frame(
  author_id      = sampled_ext$author_id[i],
  label          = sampled_ext$label[i],
  first_name     = first,
  last_name      = last,
  university     = info_df$affiliation_name,
  country        = info_df$affiliation_country,
  scopus_id      = info_df$scopus_id,
  stringsAsFactors = FALSE
)
results_list[[i]] <- combined

Sys.sleep(0.2)
}

final_results <- do.call(rbind, results_list)

write.csv(final_results, output_file, row.names = FALSE, fileEncoding = "UTF-8")
message(sprintf("Fertig! Ergebnis gespeichert in '%s'.", output_file))

```

Scopus Single Author Test

```

if (!requireNamespace("httr", quietly = TRUE)) {
  install.packages("httr")
}

```

```

if (!requireNamespace("jsonlite", quietly = TRUE)) {
  install.packages("jsonlite")
}

library(httr)
library(jsonlite)

scopus_api_key <- "fe6385c29022b7b8b8c8217cddfff7de"

fetch_scopus_for_one <- function(first_name, last_name, api_key) {
  raw_query <- paste0("AUTHLASTNAME(", last_name, ") AND AUTHFIRST(", first_name, ")")

  base_url <- "https://api.elsevier.com/content/search/author"

  response <- GET(
    url = base_url,
    query = list(
      query = raw_query,
      count = 1
    ),
    add_headers(
      `X-ELS-APIKey` = api_key,
      Accept          = "application/json"
    )
  )

  if (status_code(response) != 200) {
    warning(sprintf("HTTP-Fehler %d beim Abruf für '%s %s'.",
                    status_code(response), first_name, last_name))
    return(NULL)
  }

  text_content <- content(response, as = "text", encoding = "UTF-8")
  parsed_json <- fromJSON(text_content, flatten = TRUE)

  entries <- parsed_json[["search-results"]][["entry"]]
  if (is.null(entries) || nrow(entries) == 0) {

```

```

message(sprintf("Kein Treffer gefunden für '%s %s'.", first_name, last_name))
return(data.frame(
  scopus_id          = NA_character_,
  affiliation_name    = NA_character_,
  affiliation_country = NA_character_,
  stringsAsFactors   = FALSE
))
}

e <- entries[1, ]

full_id <- e$`dc:identifier`
scopus_id <- if (!is.null(full_id) && grepl(":", full_id)) {
  sub(".*:(.*)$", "\\1", full_id)
} else {
  NA_character_
}

aff_name <- if (!is.null(e$`affiliation-current.affiliation-name`)) {
  e$`affiliation-current.affiliation-name`
} else {
  NA_character_
}

aff_country <- if (!is.null(e$`affiliation-current.affiliation-country`)) {
  e$`affiliation-current.affiliation-country`
} else {
  NA_character_
}

return(data.frame(
  scopus_id          = scopus_id,
  affiliation_name    = aff_name,
  affiliation_country = aff_country,
  stringsAsFactors   = FALSE
))
}

```

```

first <- "Heming"
last  <- "Wang"

result <- fetch_scopus_for_one(first, last, scopus_api_key)
print(result)

```

Scopus Structure

```

if (!requireNamespace("httr", quietly = TRUE)) {
  install.packages("httr")
}

if (!requireNamespace("jsonlite", quietly = TRUE)) {
  install.packages("jsonlite")
}

library(httr)
library(jsonlite)

scopus_api_key <- "fe6385c29022b7b8b8c8217cddfff7de"
first  <- "Heming"
last   <- "Wang"

response <- GET(
  url = "https://api.elsevier.com/content/search/author",
  query = list(query = paste0("AUTHLASTNAME(", last, ") AND AUTHFIRST(", first, ")"), count = 1),
  add_headers(
    `X-ELS-APIKey` = scopus_api_key,
    Accept          = "application/json"
  )
)

raw_text  <- content(response, as = "text", encoding = "UTF-8")
parsed_all <- fromJSON(raw_text, flatten = TRUE)

entries <- parsed_all[["search-results"]][["entry"]]

```

```

cat("Anzahl Einträge:", length(entries), "\n\n")

if (!is.null(entries) && length(entries) >= 1) {
  e <- entries[1, ]
  cat("Strukturbeschreibung des ersten Eintrags\n")
  str(e)
  cat("\n Erster Datensatz\n")
  print(e)
} else {
  cat("Kein Eintrag vorhanden.\n")
}

```